URL FETCHER

A REUSABLE COMPONENT TO FETCH URLs.

Example: Discovering gov.in and nic.in URLs with crt.sh and Common Crawl

Practical handover document for repeatable, passive hostname discovery. Note: Only hostnames, no paths or pages.

Submitted by: Shaik Zaheer

1. Purpose and scope

This document records the workflow used to discover hostnames associated with the gov.in and nic.in domain suffixes. It is intended to help a new team member repeat the collection without rebuilding the approach from scratch. It documents the two approved sources in this workflow: crt.sh and Common Crawl.

What the script does

- Queries crt.sh for public certificate records matching each target suffix.
- Queries the newest available Common Crawl index and extracts hostnames from archived URLs.
- Normalizes hostname text, rejects names outside the requested suffixes, and removes duplicates.
- Combines both source sets and exports results to plain-text and Excel files.

What the output does not guarantee

A hostname in the output is a discovery result, not proof that the website is currently online, owned by a particular department, reachable from every network, or configured for HTTPS. The script prefixes results with https:// for convenient review; that prefix is not a connectivity test. Confirm findings separately before using the list operationally.

2. Sources (2) and their roles

I. crt.sh — Certificate Transparency search

crt.sh provides a searchable interface to Certificate Transparency (CT) certificate data. Certificates can list one or more DNS names, including subdomains. Searching these records can reveal hostnames that have appeared in certificate data, even if they are not easy to find through ordinary navigation. The script requests JSON, excludes expired certificates where supported by the query, asks for deduplicated records, then parses each returned name_value field. Multiple names can be separated by newlines, so each is processed separately.

II. Common Crawl — Web archive index

Common Crawl indexes URLs captured by its web crawlers. The script first retrieves the crawl collection list, then selects the newest listed snapshot (CRAWLS = 1). For each suffix, it queries the snapshot's CDX API and extracts the hostname from each returned URL. This source reflects URLs seen in crawl data and may include older or no-longer-active websites.

Why use both?

The sources are complementary: crt.sh discovers names represented in certificate records, while Common Crawl discovers names represented in archived web URLs. Their results can overlap, but neither source contains every possible hostname. Combining and deduplicating them improves coverage compared with relying on only one source.

3. End-to-end workflow

1. **Configure scope:** Set SUFFIXES to ('gov.in', 'nic.in'). Keep the suffix checks exact so unrelated domains such as example gov.in are not included. Add any Top-Level Domain (TLD).
2. **Request crt.sh records:** Send one JSON query per suffix. If a request fails after retries, continue with the other query rather than terminating the full run.
3. **Parse certificate names:** Read each row's name_value field, split it by line, normalize each candidate, validate its suffix, and store valid hostnames in a set.
4. **Find Common Crawl snapshot:** Request collinfo.json, then use the cdx-api value for the newest listed crawl snapshot.
5. **Estimate Common Crawl pages:** Ask the CDX API for showNumPages=true. Select at most MAX_PAGES pages spread across the page range rather than only taking the first pages.
6. **Parse archived URLs:** Read each JSON line, take its url field, parse the hostname with urlparse, then apply the same normalization and suffix validation.
7. **Merge source results:** Use set union (crt_hosts | crawl_hosts) to remove duplicates across the two sources.
8. **Export results:** Write a TXT file and an XLSX workbook for each suffix, sorted alphabetically, with a serial-number column in Excel.

4. Important code settings

Setting	Current value	Purpose
SUFFIXES	gov.in, nic.in	Restricts discovery to the two requested suffixes.
MAX_PAGES	200	Caps Common Crawl pages sampled per suffix and snapshot.
CRAWLS	1	Uses the newest listed Common Crawl snapshot only.
Retry attempts	4	Retries transient HTTP/network errors.
Retry wait	5 × attempt seconds	Increases the wait after each failed attempt.
Common Crawl delay	1 second	Adds a brief pause between page requests.

5. Setup and execution

- Install Python 3.10 or later.
- Install the two dependencies: python -m pip install requests openpyxl
- Save the provided script as collect_gov_nic_urls.py.
- Open a terminal in the folder containing the script.
- Run: python collect_gov_nic_urls.py

An internet connection is required to reach both public sources. Depending on source response size and the number of sampled Common Crawl pages, execution may take several minutes or longer.

6. Complete Python implementation (Part 1 of 2)

The code is included for reuse. Part 1 defines the settings, retry helper, hostname filter, and crt.sh collection function.

```
import json
import time
from urllib.parse import urlparse

import requests
from openpyxl import Workbook
from openpyxl.styles import Font

SUFFIXES = ("gov.in", "nic.in")         ## add any TLD.
MAX_PAGES = 200
CRAWLS = 1
HEADERS = {"User-Agent": "gov-nic-url-collector/3.0 (research script)"}
DOTTED = tuple("." + suffix for suffix in SUFFIXES)


def get(url, params=None, timeout=180):
    """Try a request up to four times with increasing waits."""
    for attempt in range(1, 5):
        try:
            response = requests.get(
                url, params=params, headers=HEADERS, timeout=timeout
            )
            response.raise_for_status()
            return response
        except requests.RequestException as error:
            print(f"  Attempt {attempt}/4 failed: {type(error).__name__}")
            if attempt < 4:
                time.sleep(5 * attempt)
    return None


def keep(host):
    """Normalize a hostname and keep only the target suffixes."""
    host = host.strip().lower().lstrip("*.").rstrip(".")
    if "@" not in host and (host in SUFFIXES or host.endswith(DOTTED)):
        return host
    return None


def from_crtsh():
    """Collect names listed in public Certificate Transparency records."""
    found = set()
    for suffix in SUFFIXES:
        print(f"[crt.sh] Querying *.{suffix}")
        response = get(
            "https://crt.sh/",
            params={
                "q": f"%.{suffix}",
```

6. Complete Python implementation (Part 2 of 2)

Part 2 collects Common Crawl results, exports the combined results, and runs the workflow.

```
                "output": "json",
                "exclude": "expired",
                "deduplicate": "Y",
            },
            timeout=180,
        )
        if response is None:
            continue
        try:
            rows = response.json()
        except ValueError:
            print(f"  crt.sh returned non-JSON data for {suffix}")
            continue

        for row in rows:
            for name in row.get("name_value", "").splitlines():
                hostname = keep(name)
                if hostname:
                    found.add(hostname)
    return found


def from_common_crawl():
    """Collect names found in the newest Common Crawl index."""
    found = set()
    response = get("https://index.commoncrawl.org/collinfo.json", timeout=60)
    if response is None:
        return found
    try:
        crawls = response.json()[:CRAWLS]
    except ValueError:
        print("[Common Crawl] Could not read the index list")
        return found

    for crawl in crawls:
        api = crawl.get("cdx-api")
        if not api:
            continue
        for suffix in SUFFIXES:
            print(f"[Common Crawl] Querying *.{suffix}")
            query = {"url": f"*.{suffix}", "output": "json", "fl": "url"}
            response = get(api, {**query, "showNumPages": "true"}, timeout=180)
            if response is None:
                continue
            try:
                total_pages = int(response.json().get("pages", 1))
            except (ValueError, TypeError):
                print("  Could not determine result page count")
                continue
            if total_pages <= 0:
                continue

            count = min(total_pages, MAX_PAGES)
            if count == 1:
                pages = [0]
            else:
                pages = sorted({
                    round(i * (total_pages - 1) / (count - 1))
                    for i in range(count)
                })

            for page in pages:
                response = get(api, {**query, "page": page}, timeout=180)
                if response is not None:
                    for line in response.text.splitlines():
                        try:
                            url = json.loads(line).get("url", "")
                            hostname = urlparse(url).hostname or ""
                        except (ValueError, TypeError):
                            continue
                        hostname = keep(hostname)
                        if hostname:
                            found.add(hostname)
                print(f"  Page {page} checked; {len(found)} unique hosts so far")
                time.sleep(1)
    return found
```

```
def save(hosts):
    """Export separate TXT and XLSX files for each suffix."""
    for suffix in SUFFIXES:
        urls = sorted(
            f"https://{host}" for host in hosts
            if host == suffix or host.endswith("." + suffix)
        )
        base = suffix.replace(".", "_") + "_urls"
        if not urls:
            print(f"No {suffix} URLs found; skipping {base} files.")
            continue

        with open(base + ".txt", "w", encoding="utf-8") as file:
            file.write("\\n".join(urls) + "\\n")

        workbook = Workbook()
        sheet = workbook.active
        sheet.title = suffix
        sheet.append(["S.No", "Website URL"])
        for number, url in enumerate(urls, start=1):
            sheet.append([number, url])
        for cell in sheet[1]:
            cell.font = Font(bold=True)
        sheet.freeze_panes = "A2"
        sheet.column_dimensions["B"].width = 55
        try:
            workbook.save(base + ".xlsx")
        except PermissionError:
            print(f"Close {base}.xlsx in Excel and run again.")
            continue
        print(f"Saved {len(urls)} {suffix} URLs to {base}.txt and {base}.xlsx")


if __name__ == "__main__":
    crt_hosts = from_crtsh()
    crawl_hosts = from_common_crawl()
    print(f"\\ncrt.sh: {len(crt_hosts)} | Common Crawl: {len(crawl_hosts)}")
    combined = crt_hosts | crawl_hosts
    print(f"Combined unique hostnames: {len(combined)}")
    save(combined)
    print("Finished.")
```

7. Failures, errors, and Fix

Problem	Recommended interpretation or action
Timeout, connection error, or HTTP error	The get() helper retries up to four times and waits progressively longer. If all attempts fail, it returns None and the relevant source/query is skipped. Check connectivity and rerun later; a run may be partial.
crt.sh returns non-JSON content	A service error, rate-limit page, or unexpected response may not be valid JSON. The script prints a message and skips that suffix's response rather than crashing.
Common Crawl collection list cannot be read	If collinfo.json is unreachable or invalid, Common Crawl contributes no results for that run. Retry later and check whether the public index endpoint is reachable.
Could not determine page count	The page-count response may be malformed or unavailable. The script skips that suffix for the affected snapshot. Review the response/API status before interpreting the result as empty.
Zero URLs found	This can mean the source query returned no matching records, a request failed, or parsing/filtering removed the candidates. It does not prove that no sites exist. Check the console output and rerun.
Excel PermissionError	The workbook may be open in Excel or locked by another process. Close the workbook and rerun the script.
Script is slow	Common Crawl page requests are the main time cost. MAX_PAGES controls the cap; lowering it speeds up execution but reduces coverage. Keep a respectful request rate.
Unexpected hostname included/excluded	Review keep(): it strips wildcard prefixes and trailing dots, lowercases names, and accepts only the exact suffix or a hostname ending in .gov.in / .nic.in.

8. Known limitations

- Coverage is not exhaustive: only names represented in the queried source data can be found.
- Common Crawl page sampling is capped at 200 pages per suffix and selected snapshot; some results may be missed.
- The script uses the newest listed snapshot only. Older snapshots may contain different hostnames.
- Certificate records and crawl captures may be stale, duplicated, or associated with sites that are no longer active.
- The exported URLs are constructed using https:// and are not actively checked for DNS resolution, HTTP status, redirects, or TLS validity.
- The script collects hostnames, not a complete list of paths, APIs, or pages within each website.

9. Output files

Filename	Contents
gov_in_urls.txt	One HTTPS-prefixed gov.in hostname per line.
gov_in_urls.xlsx	gov.in results with S.No and Website URL columns.
nic_in_urls.txt	One HTTPS-prefixed nic.in hostname per line.
nic_in_urls.xlsx	nic.in results with S.No and Website URL columns.

10. Quality checks after collection

- Confirm the terminal summary shows separate crt.sh and Common Crawl counts and a combined unique-host count.
- Confirm the expected TXT/XLSX files were created for suffixes with results.
- Compare the first few TXT lines with the corresponding Excel entries to ensure the exports agree.
- Check for duplicates only if modifying the script; set-based storage should already remove duplicate hostnames.
- For a sample of results, validate DNS and HTTPS reachability separately using approved methods. Record the date and validation method because reachability changes.
- If completeness is important, increase MAX_PAGES carefully or run additional crawl snapshots as a documented change; note the increased runtime and request volume.

11. Reuse checklist

- Confirm that the scope remains gov.in and nic.in.
- Install dependencies in the intended Python environment.
- Run the script and retain the console output with the result files.
- Record the run date, Python version, settings (MAX_PAGES and CRAWLS), and any skipped source/query.
- Do not describe the result as a complete inventory or a list of verified live sites.
- Only perform follow-on validation or scanning when explicitly authorized by the organization.

12. References

crt.sh — public Certificate Transparency search: https://crt.sh/

Common Crawl — index collection list: https://index.commoncrawl.org/collinfo.json

Common Crawl index API documentation: https://index.commoncrawl.org/

Implementation basis: the uploaded collect_gov_nic_urls Python script, with explanatory documentation added for handover. The guide describes the current approach and its limits; it does not claim that every query or returned record has been independently verified.

Appendix: dependency installation and run commands

```
python -m pip install requests openpyxl
python collect_gov_nic_urls.py
```

Submitted by: Shaik Zaheer